

// ИТ-СИМУЛЯТОР · Лекция

Бизнес-процессы IT-компании

*Карта мира для тех, кто только
начинает путь в программировании*

Зачем тебе это знать прямо сейчас?

1

Ты учишь синтаксис — но не контекст

Знать ``for`` и ``if`` — это 10% работы программиста. Остальные 90% — это команда, процессы и общение.

2

Чтобы не «утонуть» в первый месяц

На стажировке тебе скажут: «возьми задачу из Jira, сделай PR, выкатим в canary». Ты должен понимать каждое слово.

3

Чтобы пройти собеседование

На собесе спросят не только про код. Спросят: «Расскажи, как у вас был организован Scrum».

4

Чтобы выбрать правильное место

Не все компании одинаковы. Зная типы, ты осознанно выберешь, где начинать карьеру.

О чём мы поговорим

01

Где работают программисты

3 типа IT-компаний и их различия

02

Кто все эти люди

Роли в команде и иерархия

03

Как живёт задача

От идеи до реальных пользователей

04

Ритуалы Agile

5 встреч, которые тебя ждут

05

Как принимаются решения

Code Review, Pull Request, CI/CD

06

Твой первый день

Чек-лист и главные ошибки джунов

Главный миф о работе программиста

МИФ

**«Программист сидит
в наушниках и пишет код»**

*Тихая комната, чёрный экран, чай рядом.
8 часов кодинга. И так каждый день.*

РЕАЛЬНОСТЬ

**«Программист — это
командный игрок»**

- 30–40% времени — встречи и общение
- 30% — чтение чужого кода и review
- 20% — собственно написание кода
- 10% — отладка и поддержка прода

01

БЛОК ПЕРВЫЙ

Где работают программисты

Три типа IT-компаний

Они выглядят похоже снаружи, но устроены совсем по-разному

ПРОДУКТ

Продуктовая

Аналогия:

*Ресторан со
своей кухней*

Примеры:

Яндекс, Ozon,
Авито, Тинькофф

АУТСОРС

Аутсорс

Аналогия:

*Кейтеринг
по заказу*

Примеры:

ЕРАМ, Andersen,
веб-студии

АУТСТАФФ

Аутстафф

Аналогия:

*Повар
напрокат*

Примеры:

Ты в команде
другой компании

Продуктовая компания

«Ресторан со своей кухней»: компания делает свой продукт, на нём же зарабатывает

Что это значит для тебя

- Годами работаешь над одним продуктом
- Видишь, как растёт количество пользователей
- Релизы — часто
- Зарплаты обычно выше рынка
- Тебя оценивают по влиянию на бизнес-метрики
- Меньше бюрократии, чем в энтерпрайзе

Примеры

Яндекс · Авито · Ozon · Wildberries
Тинькофф · Miro · Notion · Vercel

Подходит тебе, если...

...хочешь глубоко погружаться в одну предметную область, видеть результат своей работы на конечных пользователях, расти технически в большой команде.

Аутсорс-компания

«Кейтеринг по заказу»: компания пишет код для других компаний по контракту (услуга под ключ)

Что это значит для тебя

- Каждые 6–18 месяцев новый проект
- Стек разнообразный: Angular → React → Vue
- Меньше эмоциональной привязки к продукту
- Жёстче с дедлайнами — заказчику обещали
- Чёткая лестница: junior → middle → senior
- Зарплаты порой ниже, чем в продукте

Примеры

EPAM · Andersen · Accenture
Тысячи мелких веб-студий

Подходит тебе, если...

...любишь разнообразие, не хочешь привязываться к одной области, хочешь быстро накопить разные кейсы в портфолио и увидеть много разных индустрий.

Аутстаф-компания

«Повар напрокат»: компания "сдаёт в аренду" своих разработчиков команде клиента

Что это значит для тебя

- Зарплату платит компания А
- Задачи получаешь от тимлида компании Б
- Полноценный член команды Б — без её бонусов
- Если проект Б закроется — ищут новый
- Между проектами «бенч» (без работы)
- Зарплата ниже, но проще войти в индустрию

Как работает экономика

Клиент платит компании \$50/час
Компания платит тебе \$25/час
Разница (\$25/час) — её прибыль

Подходит тебе, если...

...ты в начале пути и главное — попасть в реальную команду. После аутстафа большинство переходит в продуктовые компании или аутсорс.

Сводная таблица: что выбрать?

Это не «лучше или хуже», это просто разные модели — выбирай под свою цель

Критерий	Продукт	Аутсорс	Аутстафф
Зарплата	Высокая	Средняя	Ниже средней
Глубина в продукт	Очень высокая	Поверхностная	Зависит от клиента
Разнообразие стека	Низкое	Высокое	Среднее
Стабильность	Высокая	Средняя	Низкая (бенч)
Карьерный рост	По грейдам, гибко	По чётким уровням	Через смену проектов

Идеально для	Опытных и Middle+	Универсалов	Старта карьеры
--------------	-------------------	-------------	----------------

02

БЛОК ВТОРОЙ

Кто все эти люди в команде

Кто все эти люди?

В команде из 8–10 человек ты встретишь представителей минимум 5 разных профессий

01

Бизнес

- Product Manager
- Project Manager
- Заказчик

02

Разработка

- Frontend
- Backend
- Тимлид

03

Качество

- QA Manual
- QA Automation

04

Дизайн

- UX Designer
- UI Designer
- Product Designer

Бизнес-сторона: кто решает, что делать

Заказчик / Sponsor

У него идея и деньги

Волнует один вопрос: «Сколько я заработаю и когда?»
В продуктовой компании — руководство. В аутсорсе — внешний клиент.

Product Manager

Голос заказчика внутри команды

Решает ЧТО делать и ПОЧЕМУ. Ведёт бэклог задач, расставляет приоритеты, общается с пользователями.

Project Manager

Решает КАК и КОГДА доставить

Управляет сроками, рисками, зависимостями.
Важно: в современных продуктовых командах его часто НЕТ.



Подсказка для собесы: на вопрос «кому ты будешь задавать вопросы по бизнес-логике?» — правильный ответ: РМ, не тимлиду.

Тимлид: твой главный человек в команде

К нему ты идёшь и с техническими вопросами, и со всем остальным

по коду

Технический наставник

К нему идёшь, когда...

- Не понимаешь, как устроен модуль
- Нужно согласовать архитектуру
- Застрял на сложной задаче
- Нужен code review важного PR
- Не понимаешь, какую технологию выбрать

по людям и карьере

Твой руководитель

К нему идёшь, когда...

- Хочешь поднять зарплату или грейд
- Конфликт с кем-то в команде
- Нужен отпуск, больничный, отгул
- Performance review раз в полгода
- Хочешь сменить проект или команду

В крупных компаниях иногда эти две зоны разделяют между двумя людьми. У нас и в большинстве команд это один человек — тимлид.

Дизайнеры: UX и UI

UX Designer

«Как **ЛОГИКА** работает»

- Проектирует, как пользователь взаимодействует с продуктом
- Делает прототипы и схемы экранов
- Проводит исследования пользователей
- Решает, какие кнопки и куда ведут

UI Designer

«Как это **ВЫГЛЯДИТ**»

- Превращает прототипы в красивые макеты
- Подбирает цвета, шрифты, иконки
- Работает в Figma
- Создаёт компоненты для дизайн-системы

✦ В большинстве команд это один человек — *Product Designer*

QA: те, кто проверяет, что код работает

Без них релиз — это рулетка. С ними — спокойствие

Manual QA

Тестирует руками, ищет неочевидные баги.
Лезет туда, куда автотест не догадается.

QA Automation

Пишет автотесты, которые крутятся на каждый PR.
Экономит команде дни ручной проверки.

Главная ошибка джуна:

Если QA нашёл баг — это не проблема, это его работа. Не доказывай, что баг невоспроизводимый, без проверки. Конфликт с тестировщиком — самый быстрый способ испортить репутацию в команде.

Что реально делает DevOps на работе

Если коротко — обеспечивает, чтобы твой код доехал до пользователя

Настраивает CI/CD

Пайплайн, который из твоего PR собирает приложение, прогоняет тесты и катит на серверы. Без этого деплой — ручная боль.

Поддерживает серверы

Чтобы прод не падал в три ночи. Настраивает Kubernetes, базы, очереди, балансировщики, кеши.

Следит за мониторингом

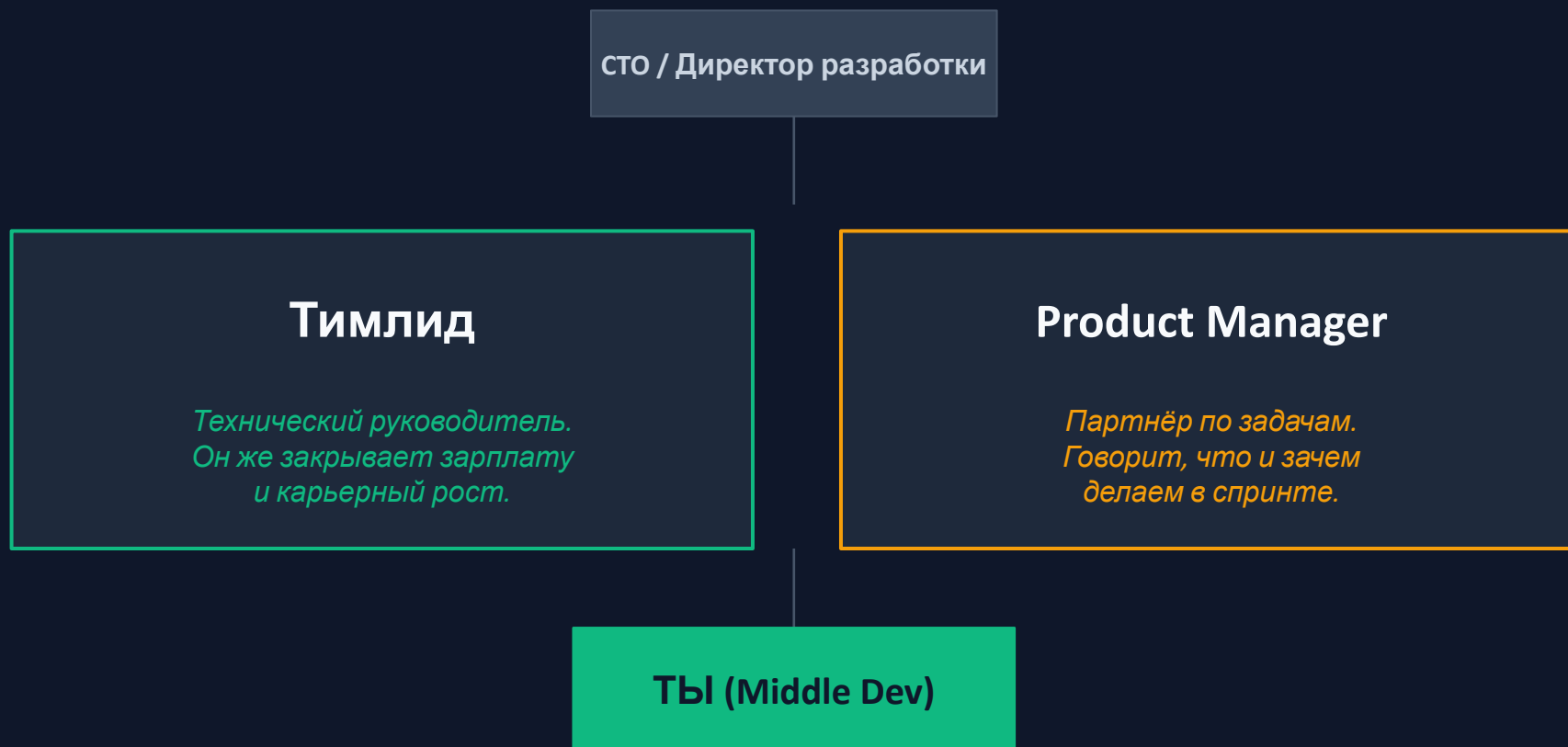
Дашборды, алерты, логи. Когда что-то ломается — DevOps первый видит и поднимает дежурного.

Помогает разработчикам

Даёт доступы, пишет инструкции, разбирается с непонятными ошибками деплоя. Твой главный союзник, когда упал прод.

Кому ты подчиняешься как middle?

В большинстве команд два главных человека для тебя — тимлид и РМ



03

БЛОК ТРЕТИЙ

Как живёт задача от идеи до прода

Две параллельные дорожки

Главное, что нужно понять про современный продуктовый процесс

ИССЛЕДОВАНИЕ

Discovery

«Стоит ли это вообще делать?»

- РМ общается с пользователями
- Дизайнер рисует прототипы
- Проверяют гипотезы
- Решают: делать / не делать

РАЗРАБОТКА

Delivery

«Как это реализовать?»

- Разработчики пишут код
- QA проверяет качество
- Релизы идут в продакшен
- Команда смотрит на метрики

Сейчас ты делаешь фичу, которую исследовали месяц назад. Тем временем продукт-команда уже исследует ту, что ты будешь делать через месяц.

От идеи до планирования

1

Идея и Discovery

Кто-то заметил проблему. РМ проводит интервью, дизайнер рисует прототип. Результат — product brief.

2

Тех-дизайн

Тимлид пишет документ: какие компоненты изменятся, какие новые API нужны, какие риски. Команда обсуждает.

3

Декомпозиция и оценка

Большая фича режется на мелкие задачи в Jira. Команда оценивает каждую в story points (1, 2, 3, 5, 8, 13).

4

Планирование спринта

В начале 2-недельного спринта команда выбирает задачи на ближайшие 2 недели с учётом своей velocity.

 *story points — относительная сложность, шкала Фибоначчи. velocity — сколько points команда закрывает за спринт.*

Разработка, ревью, тестирование

5

Разработка

Берёшь задачу, переводишь в «In Progress», создаёшь ветку от main, пишешь код..

6

Code Review

Создаешь Pull Request. Минимум один человек должен его одобрить. Автоматически крутятся тесты и линтер.

7

Тестирование

QA проверяет руками, или автотесты гоняются автоматически. Если найден баг — задача возвращается на доработку.

 Pull Request (PR) — запрос на слияние твоего кода в основную ветку. Линтер — программа, проверяющая стиль кода.

Релиз и мониторинг

8

Релиз

Код мерджится в main. Дальше CI/CD-пайплайн собирает приложение, прогоняет тесты на staging и катит на прод.

В зрелых командах от мержа до прода — 15–30 минут.

9

Мониторинг и поддержка

После релиза команда следит за здоровьем фичи. Не «закодил и забыл», а:

- смотрят на дашборды
- проверяют error rate
- следят за бизнес-метриками
- при проблеме — откат

Подождите. В реальности всё не так гладко

В жизни задача проходит этапы НЕ линейно

- Задачи блокируют друг друга — ждёшь, пока бэкендер сделает API
- Тех-дизайн меняется в процессе разработки
- На code review находят проблемы — переписываешь половину
- QA находит баги — задача возвращается на доработку
- Прод сломался — весь спринт ломается

✦ *Не паникуй, что задачи висят. Это нормально. Главное — движение, а не застревание в одном месте.*

04

БЛОК ЧЕТВЁРТЫЙ

**5 ритуалов Agile,
которые тебя ждут**

Что такое Scrum простыми словами?

Scrum — это способ работы команды короткими циклами по 2 недели

Каждый цикл (спринт) команда выбирает себе задачи, делает их, потом показывает результат и обсуждает, что улучшить.



Один спринт = 2 недели = 10 рабочих дней

1. Daily Standup

Ежедневная встреча команды на 15 минут — каждое утро

3 вопроса, на которые отвечает каждый

1. Что сделал вчера
2. Что планирую сегодня
3. Что меня блокирует

Что говорить как middle:

«Закончил JIRA-1234, сегодня начинаю JIRA-1245. Меня блокирует Иван — жду от него обновлённый API»

Антипаттерны (так не делай)

- ✗ Долгие технические обсуждения внутри стендапа — выноси в отдельную встречу
- ✗ Отчёт менеджеру «я работал» — стендап не для менеджера, а для команды
- ✗ «Делаю задачу X» три дня подряд без прогресса — это сигнал, объясни затык
- ✗ Опаздывать или пропускать без предупреждения

2. Sprint Planning

1–2 часа в начале спринта — выбираете задачи на ближайшие 2 недели

- ▶ РМ представляет приоритетные задачи
- ▶ Команда обсуждает: что понятно, что неясно
- ▶ Команда оценивает задачи в story points
- ▶ Команда коммитится на объём, который реально сделает

⚠ *Главное правило для middle: не молчи. Если задача непонятна — задавай вопросы СЕЙЧАС, не через неделю в одиночку.*

3. Backlog Refinement (Grooming)

1 час раз в неделю — смотрите на задачи, которые БУДУТ делаться

Что происходит на встрече

- Уточняем требования
- Декомпозируем большие задачи
- Оцениваем в story points
- Отказываемся от неактуальных задач

Зачем это нужно?

Чтобы на Sprint Planning не было задач, в которых «надо что-то сделать, а что именно — непонятно».

Definition of Ready

Когда задача попадает на планирование, она должна быть готова к работе: описание, критерии приёмки, оценка.

4. Sprint Review (Demo)

30–60 минут в конце спринта — показываете, что сделали

✓ ХОРОШО

Показывай ЦЕННОСТЬ для пользователя

«Теперь пользователь может фильтровать заказы по дате — это ускоряет поиск в 5 раз»

Стейкхолдеры понимают, что это им даёт. Это твой шанс показать руководству свою работу.

✗ ПЛОХО

Не сыпь техническими деталями

«Я добавил RxJS оператор switchMap для обработки запросов с использованием BehaviorSubject...»

Бизнесу всё равно, как это устроено внутри. Их интересует результат для пользователей.

5. Sprint Retrospective

30–60 минут — только команда, без стейкхолдеров. Безопасное пространство



Что прошло хорошо

*Что радовало команду
и стоит продолжать
делать дальше*



Что прошло плохо

*Что мешало работать
и раздражало —
без наезда на людей*



Что меняем дальше

*Конкретные действия
на следующий спринт.
С ответственным*

Главное правило: «я-сообщения», а не «обвинения»

ХОРОШО: «Когда задачи дают без описания, мне приходится тратить день на уточнения»

ПЛОХО: «Иван не пишет нормальные тикеты»

Сколько времени уходит на встречи?

На 2-недельный спринт ритуалы съедают 5–7 часов

Ритуал	Частота	Длительность	Кто участвует
Daily Standup	Каждый день	15 минут	Команда
Sprint Planning	Раз в спринт	1–2 часа	Команда + РМ
Refinement	Раз в неделю	1 час	Команда + РМ
Demo / Review	Раз в спринт	30–60 мин	+ стейкхолдеры
Retrospective	Раз в спринт	30–60 мин	Только команда



Итого: 5–7 часов на ритуалы + 2–4 часа на рабочие созвоны = 5–10 часов в неделю на встречи

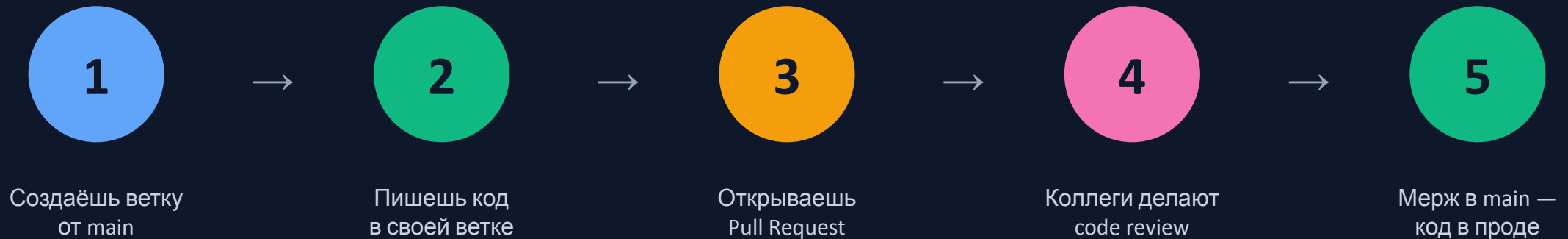
05

БЛОК ПЯТЫЙ

Как принимаются решения в коде

Что такое Pull Request (PR)?

Главный механизм совместной работы программистов



Что проверяется на code review

Корректность логики · Читаемость кода · Соответствие стандартам команды · Безопасность · Производительность · Наличие тестов

Этикет code review: 3 категории комментариев

must-fix

Обязательно исправить

Без этого не аппрувлю.
Критичные ошибки логики, безопасности.

should-fix

Хорошо бы исправить

Не блокер, но улучшит код.
Читаемость, имена переменных.

nit / suggestion

Мелочь, на твой выбор

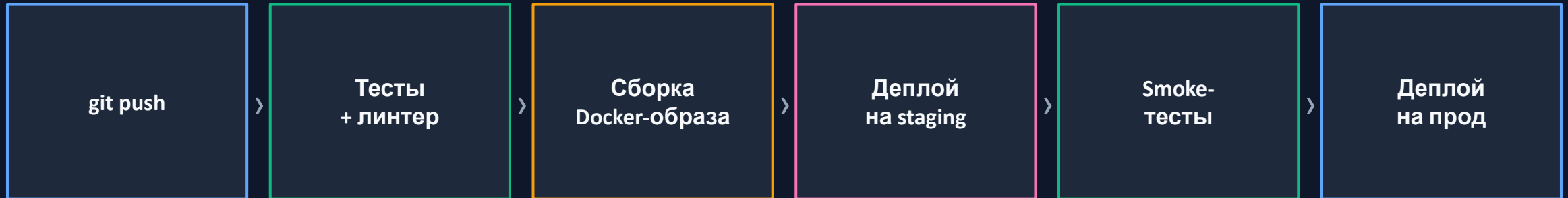
Отступы, пробелы, стиль.
Можешь не исправлять.



Хороший PR — до 400 строк изменений. Большой PR = плохой PR (ревьюить 1500 строк никто не будет).

CI/CD простыми словами

Автоматический «конвейер», который проверяет твой код и катит его в прод



⚡ В зрелой команде: от мержа PR до прода — 30–60 минут

Раньше релизы катили раз в квартал по пятницам вечером. Сейчас — десятки раз в день. Если на собесе скажешь «катали по пятницам» — посмотрят как на динозавра.

06

БЛОК ШЕСТОЙ

Твой первый день и первые 90 дней

Первая неделя на работе

✓ ЧТО ДЕЛАТЬ

- Получить доступы ко всем системам
- Запустить проект локально
- Прочитать README и onboarding-доки
- Сделать первый PR (можно на опечатку!)
- 1-on-1 с тимлидом
- 1-on-1 с PM
- Завести документ «вопросы для уточнения»

✗ ЧЕГО НЕ ДЕЛАТЬ

- Критиковать код (даже если он плохой)
- Предлагать переписать на новую технологию
- Молчать в углу — это путь джуна
- Просидеть 2 недели и сделать 0 PR
- Притворяться, что знаешь термины
- Игнорировать процессы команды

5 ошибок, которые портят репутацию

1

Боязнь задавать вопросы

Сидишь 3 дня над задачей вместо того, чтобы спросить за 5 минут. Это маркер джуна, а не middle.

2

Слишком много вопросов без подготовки

«Как это работает?» — плохой вопрос. «Я посмотрел функцию X, кажется, она делает Y, но не уверен в случае Z» — хороший.

3

Притворство, что знаешь

На митинге кивнул, потом гуглил полдня. Лучше честно сказать «не знаю этот термин, расскажи?»

4

Игнорирование процессов

Не оформил PR по шаблону, не указал тикет, не написал тесты — это сразу испортит репутацию.

5

Конфликт с тестировщиком

Если QA нашёл баг — это не проблема, это его работа. Не доказывай, что баг невоспроизводимый.

7 главных мыслей, которые ты уносишь

- 1 IT-компания — это не «программисты пишут код», а сложная система ролей и процессов.
- 2 Тимлид — твой главный человек. И по коду, и по карьере, и по конфликтам.
- 3 Задача проходит ~9 этапов от идеи до прода. Код — только один из них.
- 4 Agile — это про короткие циклы и быструю обратную связь, а не про тонну митингов.
- 5 Pull Request и code review — главный механизм совместной работы. Учись принимать критику.
- 6 DevOps в команде — твой союзник. Без него релизы — это боль и ручной труд.
- 7 Первые 90 дней критичны для твоей репутации. Не молчи, не критикуй, делай первый PR.